

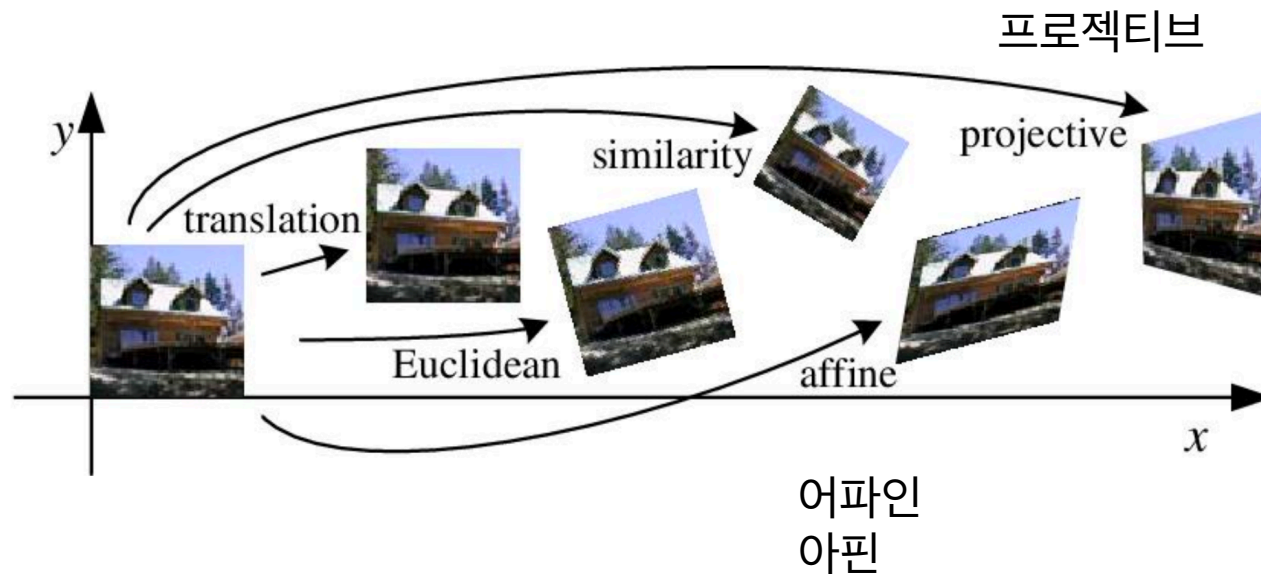
기하학적 변환

1) 영상의 이동 변환과 전단 변환

학습목표

- 영상의 기하학적 변환
- 영상의 확대와 축소
- 이미지 피라미드
- 이미지 회전
- 어파인 변환과 투시 변환
- 실전코딩 : 문서 스캐너

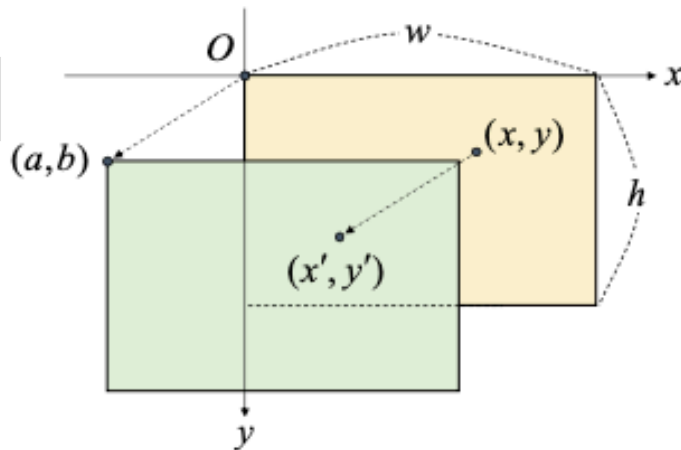
- 영상의 기하학적 변환(geometric transformation)이란?
 - 영상을 구성하는 픽셀의 배치 구조를 변경함으로써 전체 영상의 모양을 바꾸는 작업
 - Image registration, removal of geometric distortion, etc.



이동 변환(Translation transformation)

- 가로 또는 세로 방향으로 영상을 특정 크기만큼 이동시키는 변환
- x축과 y축 방향으로의 이동 변위를 지정

행렬형태로 변환



$$\begin{cases} x' = x + a \\ y' = y + b \end{cases} \quad \begin{bmatrix} x' \\ y' \end{bmatrix} = \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix} \begin{bmatrix} x \\ y \end{bmatrix} + \begin{bmatrix} a \\ b \end{bmatrix}$$

단위행렬

$$\begin{bmatrix} x' \\ y' \end{bmatrix} = \begin{bmatrix} 1 & 0 & a \\ 0 & 1 & b \end{bmatrix} \begin{bmatrix} x \\ y \\ 1 \end{bmatrix}$$

2x3 어파인 변환 행렬

영상의 이동 변환

■ 영상의 어파인 변환 함수

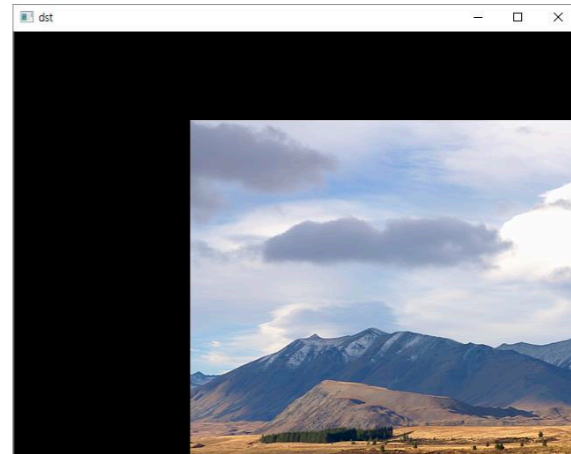
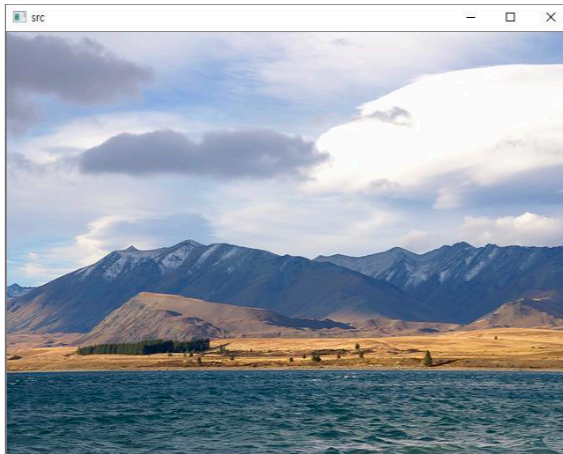
```
cv2.warpAffine(src, M, dsize, dst=None, flags=None,  
               borderMode=None, borderValue=None) -> dst
```

- src: 입력 영상
- M: 2x3 어파인 변환 행렬. 실수형.
- dsize: 결과 영상 크기. (w, h) 튜플. (0, 0)이면 src와 같은 크기로 설정.
- dst: 출력 영상
- flags: 보간법. 기본값은 cv2.INTER_LINEAR.
- borderMode: 가장자리 픽셀 확장 방식. 기본값은 cv2.BORDER_CONSTANT.
- borderValue: cv2.BORDER_CONSTANT일 때 사용할 상수 값. 기본값은 0.

영상의 이동 변환

■ 영상의 이동 변환 예제

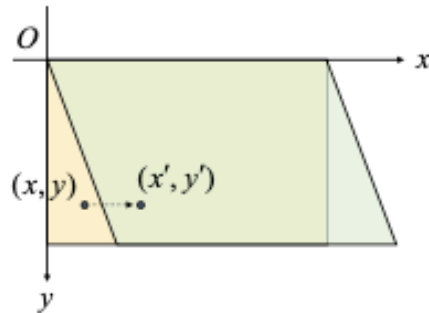
```
src = cv2.imread('tekapo.bmp')  
  
aff = np.array([[1, 0, 200],  
                [0, 1, 100]], dtype=np.float32)  
  
dst = cv2.warpAffine(src, aff, (0, 0))
```



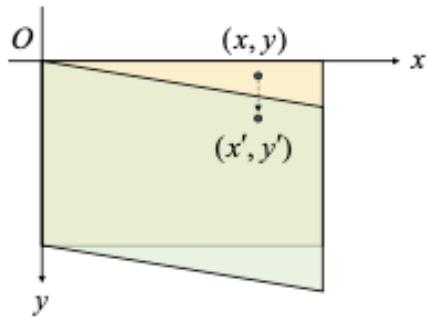
가로 200, 세로 100 이동

■ 전단 변환 (Shear transformation)

- 층 밀림 변환. x축과 y축 방향에 대해 따로 정의.



$$\begin{cases} x' = x + my \\ y' = y \end{cases} \quad \begin{bmatrix} x' \\ y' \end{bmatrix} = \begin{bmatrix} 1 & m & 0 \\ 0 & 1 & 0 \end{bmatrix} \begin{bmatrix} x \\ y \\ 1 \end{bmatrix}$$

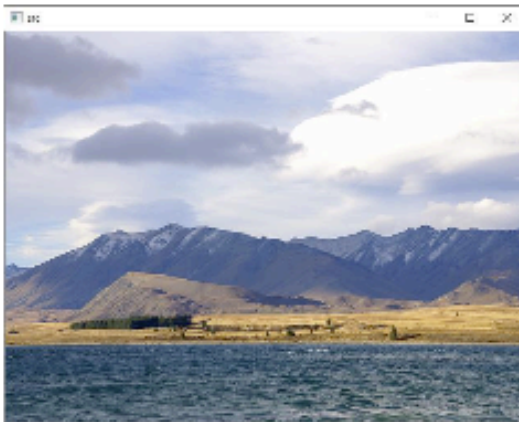


$$\begin{cases} x' = x \\ y' = mx + y \end{cases} \quad \begin{bmatrix} x' \\ y' \end{bmatrix} = \begin{bmatrix} 1 & 0 & 0 \\ m & 1 & 0 \end{bmatrix} \begin{bmatrix} x \\ y \\ 1 \end{bmatrix}$$

영상의 이동 변환

■ 영상의 전단 변환 예제

```
src = cv2.imread('tekapo.bmp')  
  
aff = np.array([[1, 0.5, 0], [0, 1, 0]], dtype=np.float32)  
  
h, w = src.shape[:2]  
dst = cv2.warpAffine(src, aff, (w + int(h * 0.5), h))
```



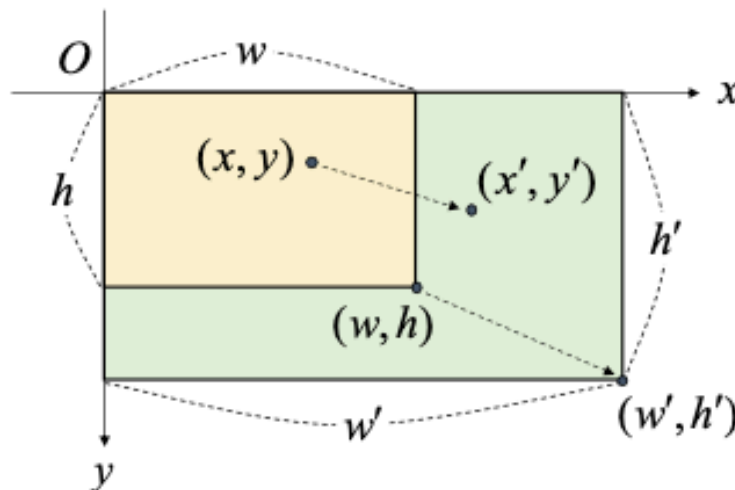
```
# dst = cv2.warpAffine(src, aff, (0,0))
```

기하학적 변환

2) 영상의 확대와 축소

■ 크기 변환(Scale transformation)

- 영상의 크기를 원본 영상보다 크게 또는 작게 만드는 변환
- x축과 y축 방향으로의 스케일 비율(scale factor)를 지정



$$\begin{cases} x' = s_x x \\ y' = s_y y \end{cases} \quad \begin{cases} s_x = w' / w \\ s_y = h' / h \end{cases}$$

$$\begin{bmatrix} x' \\ y' \end{bmatrix} = \begin{bmatrix} s_x & 0 & 0 \\ 0 & s_y & 0 \end{bmatrix} \begin{bmatrix} x \\ y \\ 1 \end{bmatrix}$$

영상의 확대와 축소

■ 영상의 크기 변환

```
cv2.resize(src, dsize, dst=None, fx=None, fy=None, interpolation=None) -> dst
```

- src: 입력 영상
- dsize: 결과 영상 크기. (w, h) 튜플. (0, 0)이면 fx와 fy 값을 이용하여 결정.
- dst: 출력 영상
- fx, fy: x와 y방향 스케일 비율(scale factor). (dsize 값이 0일 때 유효)
- interpolation: 보간법 지정. 기본값은 cv2.INTER_LINEAR

	cv2.INTER_NEAREST	최근방 이웃 보간법
	cv2.INTER_LINEAR	양선형 보간법 (2x2 이웃 픽셀 참조)
	cv2.INTER_CUBIC	3차회선 보간법 (4x4 이웃 픽셀 참조)
	cv2.INTER_LANCZOS4	Lanczos 보간법 (8x8 이웃 픽셀 참조)
	cv2.INTER_AREA	영상 축소 시 효과적



영상의 확대와 축소

■ 영상의 크기 변환 예제

default

```
src = cv2.imread('rose.bmp') # 480x320

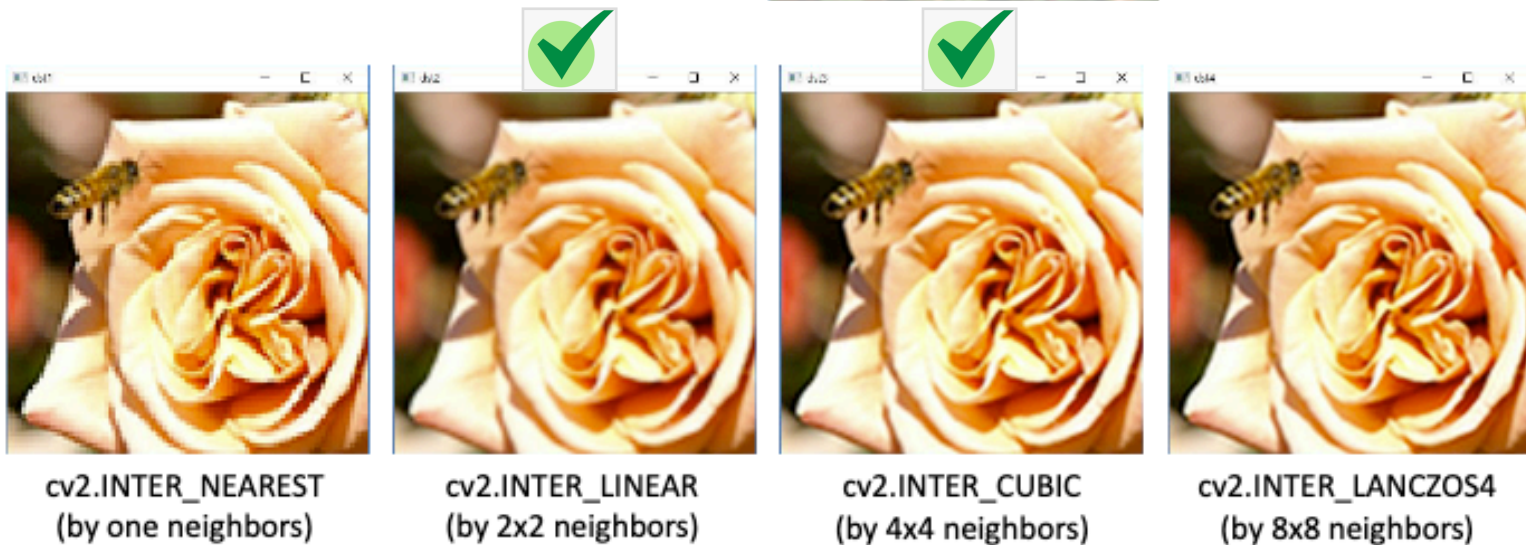
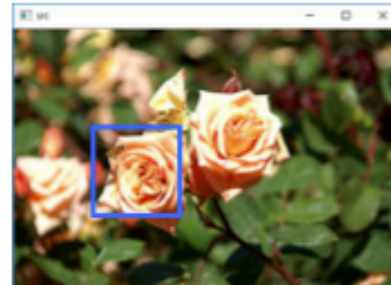
dst1 = cv2.resize(src, (0, 0), fx=4, fy=4, interpolation=cv2.INTER_NEAREST)
dst2 = cv2.resize(src, (1920, 1280)) # cv2.INTER_LINEAR
dst3 = cv2.resize(src, (1920, 1280), interpolation=cv2.INTER_CUBIC)
dst4 = cv2.resize(src, (1920, 1280), interpolation=cv2.INTER_LANCZOS4)

cv2.imshow('src', src)
cv2.imshow('dst1', dst1[500:900, 400:800])
cv2.imshow('dst2', dst2[500:900, 400:800])
cv2.imshow('dst3', dst3[500:900, 400:800])
cv2.imshow('dst4', dst4[500:900, 400:800])
cv2.waitKey()

cv2.destroyAllWindows()
```


영상의 확대와 축소

■ 영상의 크기 변환 예제 결과



cv2.INTER_NEAREST
(by one neighbors)

cv2.INTER_LINEAR
(by 2x2 neighbors)

cv2.INTER_CUBIC
(by 4x4 neighbors)

cv2.INTER_LANCZOS4
(by 8x8 neighbors)

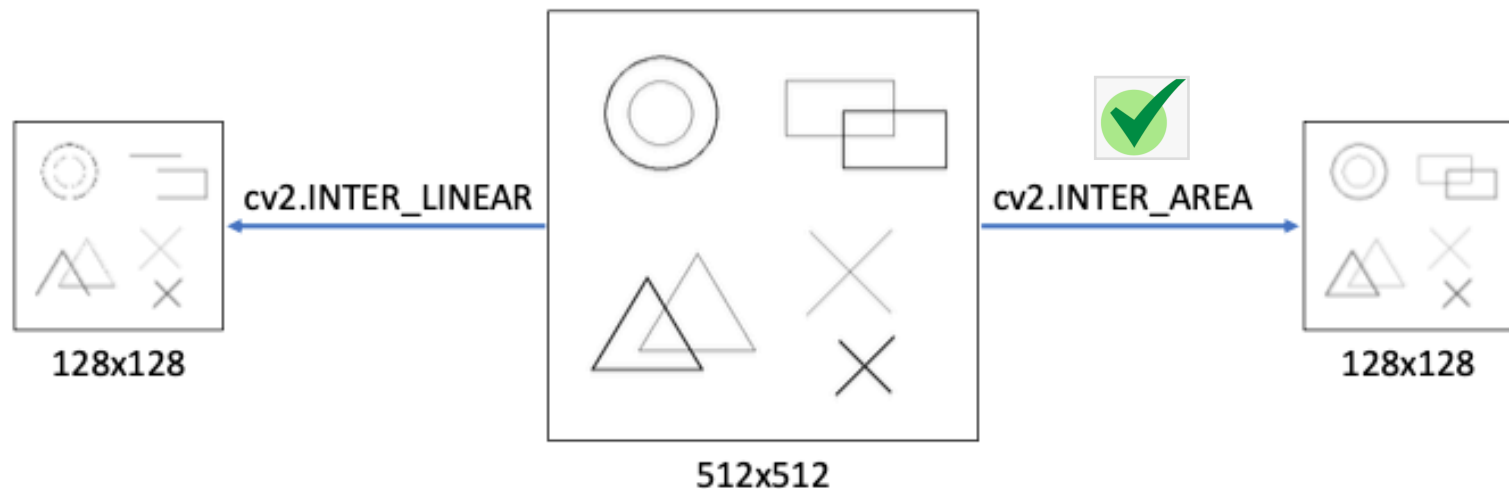
계단현상
블록현상

속도가 많이 걸림

영상의 확대와 축소

영상의 축소 시 고려할 사항

- 영상 축소 시 디테일이 사라지는 경우가 발생 (e.g. 한 픽셀로 구성된 선분)
- 입력 영상을 부드럽게 필터링한 후 축소, 다단계 축소
- OpenCV의 `cv2.resize()` 함수에서는 `cv2.INTER_AREA` 플래그를 사용



영상의 대칭

■ 영상의 대칭 변환 (flip, reflection)



좌우 대칭



상하 대칭



좌우&상하 대칭



양수 (+1)	좌우 대칭
0	상하 대칭
음수 (-1)	좌우 & 상하 대칭

영상의 대칭

■ 영상의 대칭 변환

```
cv2.flip(src, flipCode, dst=None) -> dst
```

- src: 입력 영상
- flipCode: 대칭 방향 지정

양수 (+1)	좌우 대칭
0	상하 대칭
음수 (-1)	좌우 & 상하 대칭

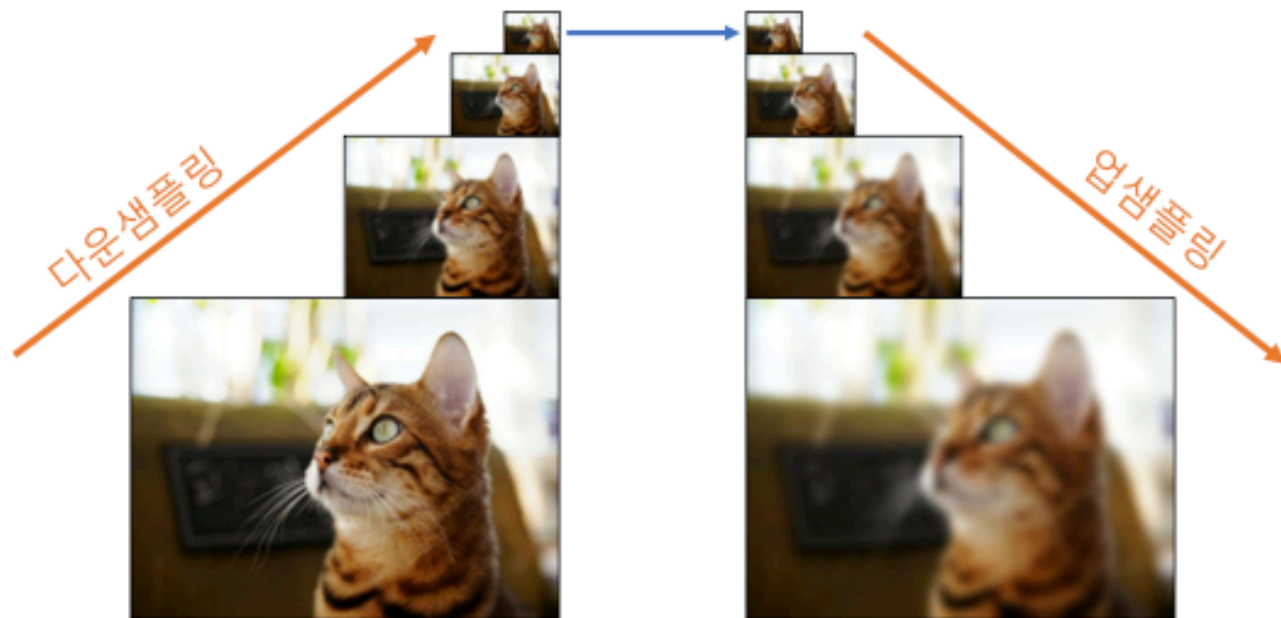
- dst: 출력 영상

기하학적 변환

3) 이미지 피라미드

이미지 피라미드

- 이미지 피라미드(Image pyramid)란?
 - 하나의 영상에 대해 다양한 해상도의 영상 세트를 구성한 것
 - 보통 가우시안 블러링 & 다운샘플링 형태로 축소하여 구성



이미지 피라미드

■ 영상 피라미드 다운샘플링

```
cv2.pyrDown(src, dst=None, dstsize=None, borderType=None) -> dst
```

- src: 입력 영상
- dst: 출력 영상
- dstsize: 출력 영상 크기.
 따로 지정하지 않으면 입력 영상의 가로, 세로 크기의 1/2로 설정.
- borderType 가장자리 픽셀 확장 방식
- 참고 사항
 - 먼저 5x5 크기의 가우시안 필터를 적용
 - 이후 짝수 행과 열을 제거하여 작은 크기의 영상을 생성

이미지 피라미드

■ 영상 피라미드 업샘플링

```
cv2.pyrUp(src, dst=None, dstsize=None, borderType=None) -> dst
```

- src: 입력 영상
- dst: 출력 영상
- dstsize: 출력 영상 크기.
 따로 지정하지 않으면 입력 영상의 가로, 세로 크기의 2배 설정.
- borderType 가장자리 픽셀 확장 방식

■ 피라미드 영상에 사각형 그리기 예제

```
src = cv2.imread('cat.bmp')

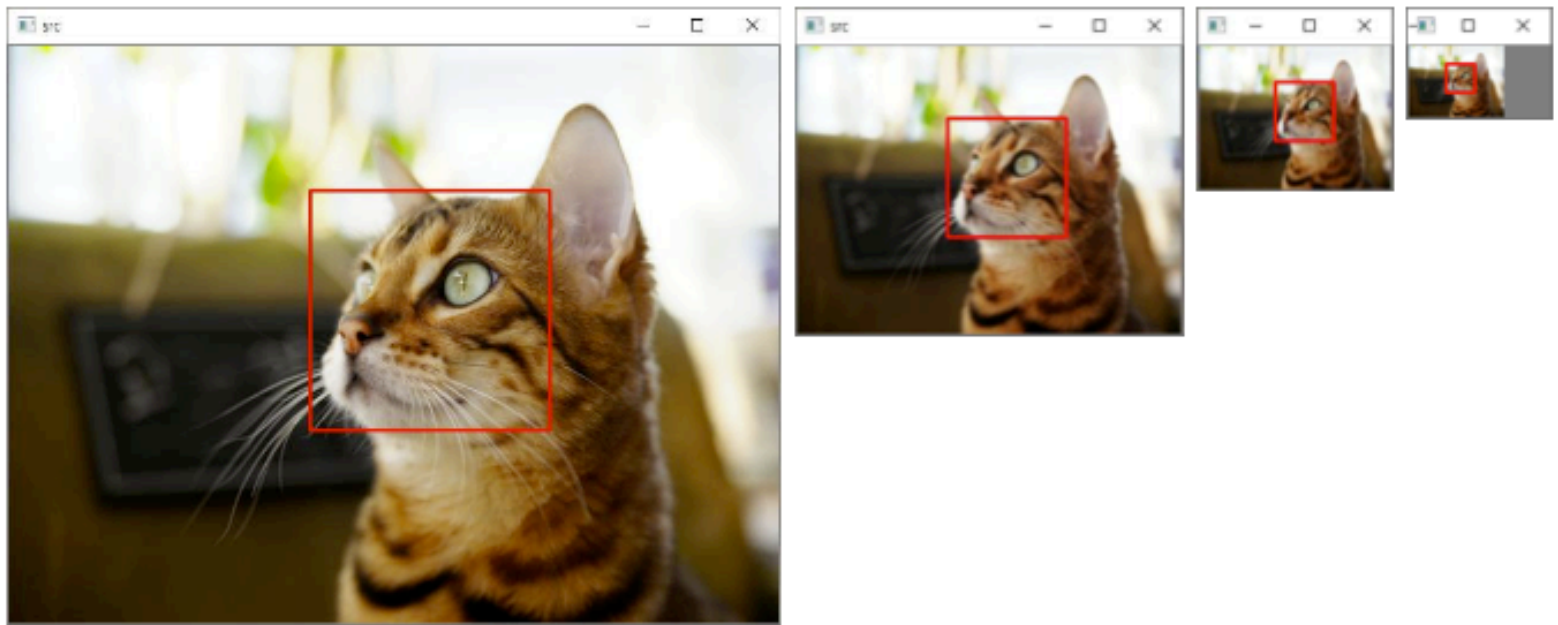
rc = (250, 120, 200, 200) # rectangle tuple

cpy = src.copy()
cv2.rectangle(cpy, rc, (0, 0, 255), 2)
cv2.imshow('src', cpy)
cv2.waitKey()

for i in range(1, 4):
    src = cv2.pyrDown(src)
    cpy = src.copy()
    cv2.rectangle(cpy, rc, (0, 0, 255), 2, shift=i)
    cv2.imshow('src', cpy)
    cv2.waitKey()
```

이미지 피라미드

- 피라미드 영상에 사각형 그리기 예제 실행 결과

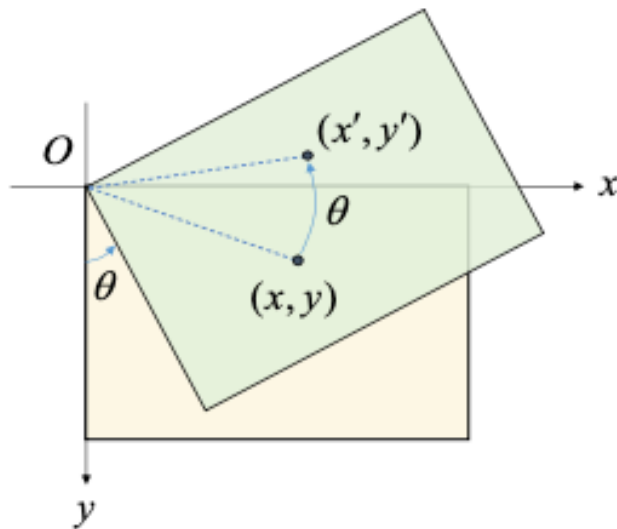


기하학적 변환

4) 영상의 회전

영상의 회전

- 회전 변환(rotation transformation)
 - 영상을 특정 각도만큼 회전시키는 변환 (반시계 방향)



$$\begin{cases} x' = \cos \theta \cdot x + \sin \theta \cdot y \\ y' = -\sin \theta \cdot x + \cos \theta \cdot y \end{cases}$$

$$\begin{bmatrix} x' \\ y' \end{bmatrix} = \begin{bmatrix} \cos \theta & \sin \theta & 0 \\ -\sin \theta & \cos \theta & 0 \end{bmatrix} \begin{bmatrix} x \\ y \\ 1 \end{bmatrix}$$

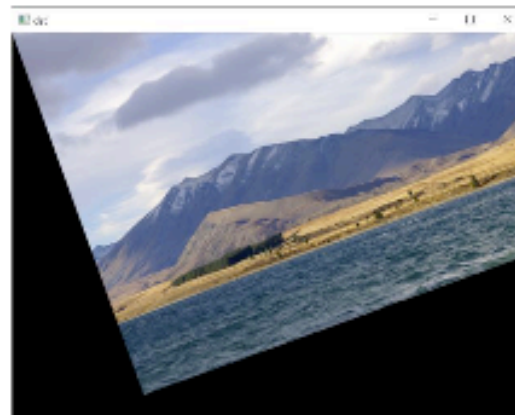
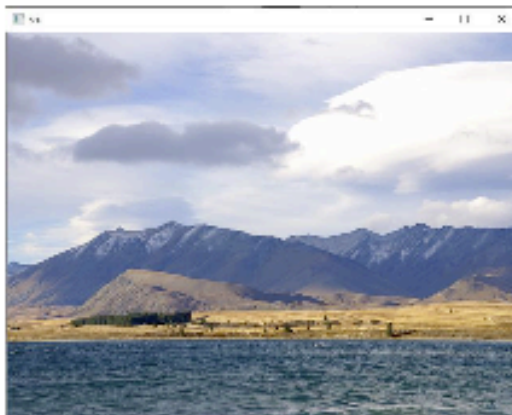
영상의 회전

■ 영상의 회전 예제

```
src = cv2.imread('tekapo.bmp')

rad = 20 * math.pi / 180
aff = np.array([[math.cos(rad), math.sin(rad), 0],
                [-math.sin(rad), math.cos(rad), 0]], dtype=np.float32)

dst = cv2.warpAffine(src, aff, (0, 0))
```



영상의 회전

영상의 회전 변환 행렬 구하기

```
cv2.getRotationMatrix2D(center, angle, scale) -> retval
```

- center: 회전 중심 좌표. (x, y) 튜플.
- angle: (반시계 방향) 회전 각도(degree). 음수는 시계 방향.
- scale: 추가적인 확대 비율
- retval: 2x3 어파인 변환 행렬. 실수형.

$$\begin{bmatrix} \alpha & \beta & (1-\alpha) \cdot \text{center.x} - \beta \cdot \text{center.y} \\ -\beta & \alpha & \beta \cdot \text{center.x} + (1-\alpha) \cdot \text{center.y} \end{bmatrix} \text{ where } \begin{cases} \alpha = \text{scale} \cdot \cos(\text{angle}) \\ \beta = \text{scale} \cdot \sin(\text{angle}) \end{cases}$$

영상의 회전

■ 영상의 중앙 기준 회전 예제

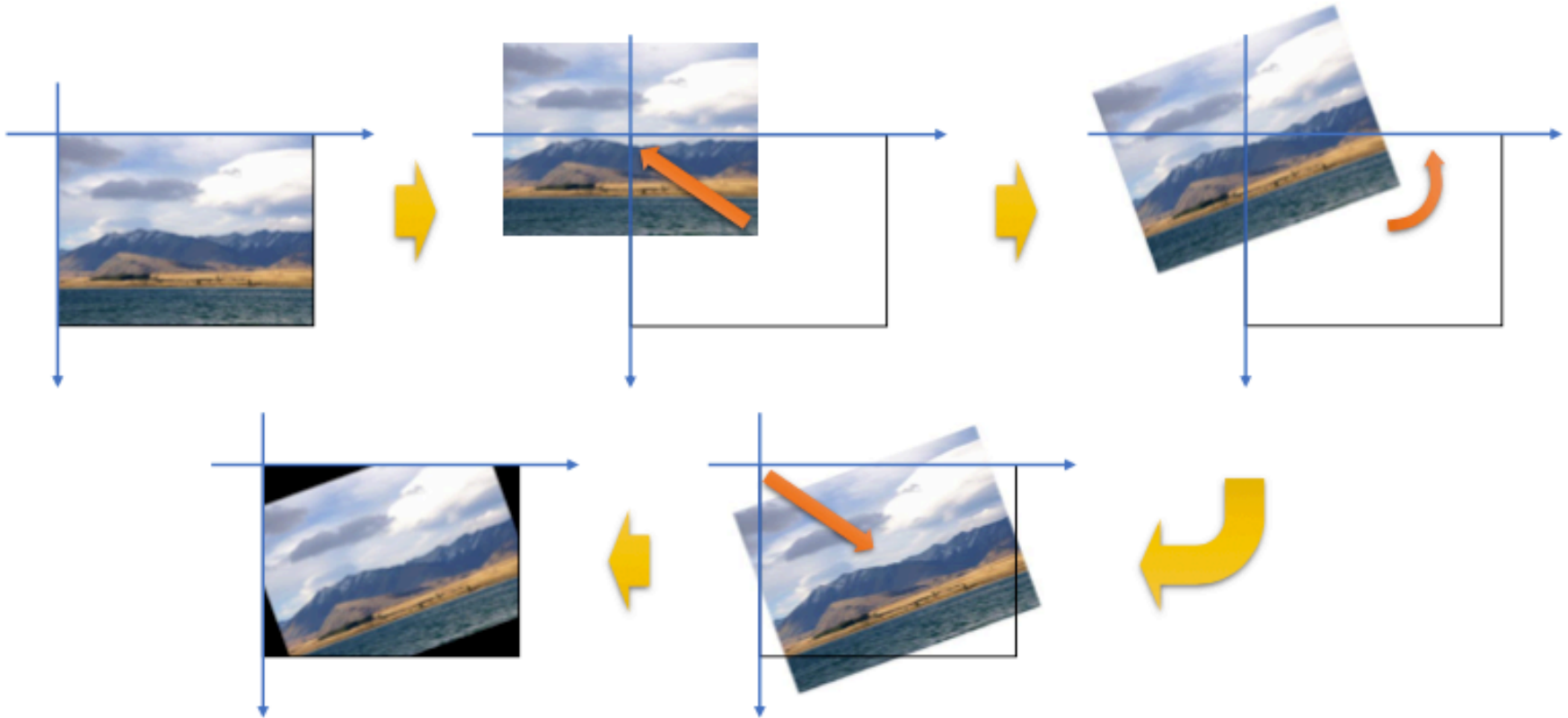
```
src = cv2.imread('tekapo.bmp')  
  
cp = (src.shape[1] / 2, src.shape[0] / 2)  
rot = cv2.getRotationMatrix2D(cp, 20, 1)  
  
dst = cv2.warpAffine(src, rot, (0, 0))
```



```
# rot = cv2.getRotationMatrix2D(cp, 20, 0.5)
```

영상의 회전

■ 영상의 중앙 기준 회전 예제 동작

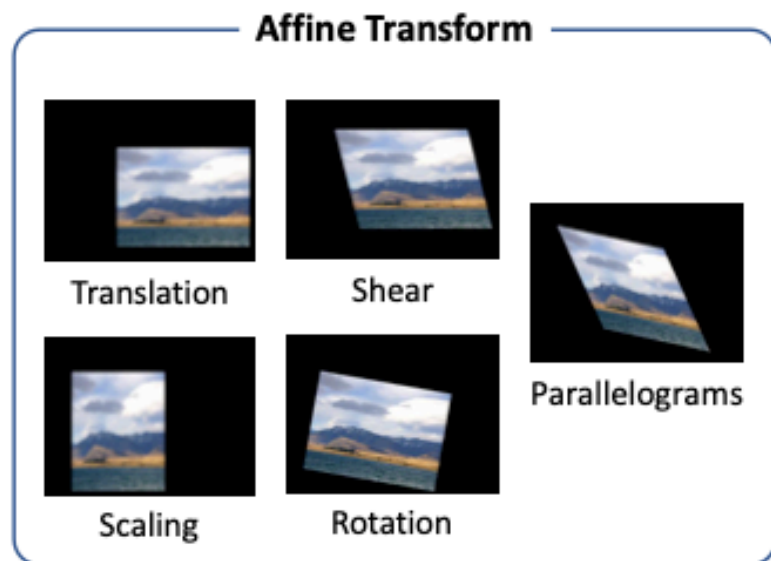


기하학적 변환

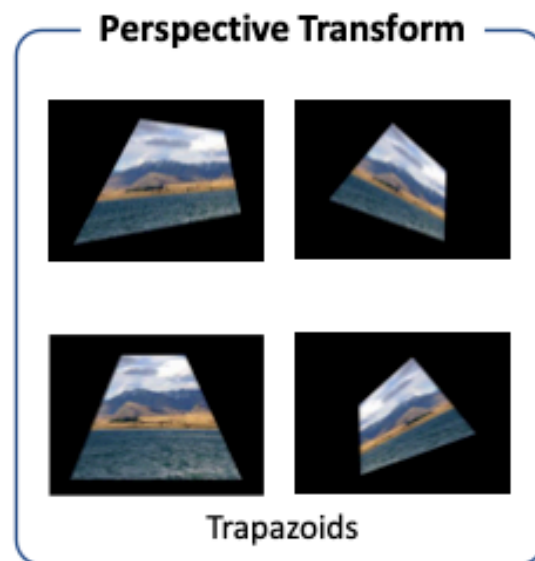
5) 어파인 변환과 투시 변환

어파인 변환과 투시 변환

■ 어파인 변환 vs. 투시 변환



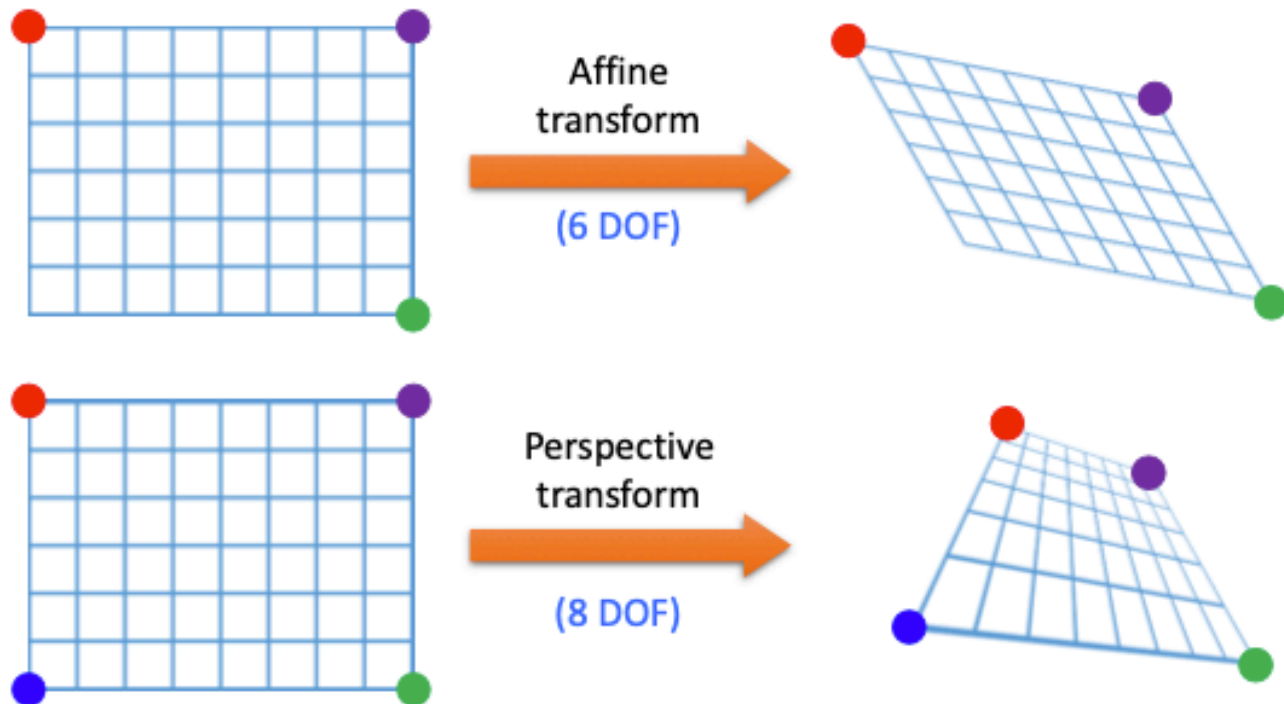
$$\begin{pmatrix} x' \\ y' \\ 1 \end{pmatrix} = \begin{bmatrix} a_{11} & a_{12} & a_{13} \\ a_{21} & a_{22} & a_{23} \\ 0 & 0 & 1 \end{bmatrix} \begin{pmatrix} x \\ y \\ 1 \end{pmatrix} \quad \text{2x3 matrix (6 DOF)}$$



$$\begin{pmatrix} wx' \\ wy' \\ w \end{pmatrix} = \begin{bmatrix} p_{11} & p_{12} & p_{13} \\ p_{21} & p_{22} & p_{23} \\ p_{31} & p_{32} & 1 \end{bmatrix} \begin{pmatrix} x \\ y \\ 1 \end{pmatrix} \quad \text{3x3 matrix (8 DOF)}$$

어파인 변환과 투시 변환

■ 어파인 변환 vs. 투시 변환



어파인 변환과 투시 변환

■ 어파인 변환 행렬 구하기

```
cv2.getAffineTransform(src, dst) -> retval
```

- src: 3개의 원본 좌표점. numpy.ndarray. shape=(3, 2)
e.g) np.array([[x₁, y₁], [x₂, y₂], [x₃, y₃]], np.float32)
- dst: 3개의 결과 좌표점. numpy.ndarray. shape=(3, 2)
- retval: 2x3 투시 변환 행렬

$$\begin{bmatrix} x'_i \\ y'_i \end{bmatrix} = \text{map_matrix} \cdot \begin{bmatrix} x_i \\ y_i \\ 1 \end{bmatrix}$$

where $dst(i) = (x'_i, y'_i)$, $src(i) = (x_i, y_i)$, $i = 0, 1, 2$

어파인 변환과 투시 변환

■ 투시 변환 행렬 구하기

```
cv2.getPerspectiveTransform(src, dst, solveMethod=None) -> retval
```

- src: 4개의 원본 좌표점. numpy.ndarray. shape=(4, 2)
e.g) np.array([[x₁, y₁], [x₂, y₂], [x₃, y₃], [x₄, y₄]], np.float32)
- dst: 4개의 결과 좌표점. numpy.ndarray. shape=(4, 2)
- retval: 3x3 투시 변환 행렬

$$\begin{bmatrix} t_i x'_i \\ t_i y'_i \\ t_i \end{bmatrix} = \text{map_matrix} \cdot \begin{bmatrix} x_i \\ y_i \\ 1 \end{bmatrix}$$

where $dst(i) = (x'_i, y'_i)$, $src(i) = (x_i, y_i)$, $i = 0, 1, 2, 3$

어파인 변환과 투시 변환

■ 영상의 어파인 변환 함수

```
cv2.warpAffine(src, M, dsize, dst=None, flags=None,
               borderMode=None, borderValue=None) -> dst
```

- src: 입력 영상
- M: 2x3 어파인 변환 행렬. 실수형.
- dsize: 결과 영상 크기. (w, h) 튜플. (0, 0)이면 src와 같은 크기로 설정.
- dst: 출력 영상
- flags: 보간법. 기본값은 cv2.INTER_LINEAR.
- borderMode: 가장자리 픽셀 확장 방식. 기본값은 cv2.BORDER_CONSTANT.
- borderValue: cv2.BORDER_CONSTANT일 때 사용할 상수 값. 기본값은 0.

어파인 변환과 투시 변환

■ 영상의 투시 변환 함수

```
cv2.warpPerspective(src, M, dsize, dst=None, flags=None,  
                    borderMode=None, borderValue=None) -> dst
```

- src: 입력 영상
- M: 3x3 투시 변환 행렬. 실수형.
- dsize: 결과 영상 크기. (w, h) 튜플. (0, 0)이면 src와 같은 크기로 설정.
- dst: 출력 영상
- flags: 보간법. 기본값은 cv2.INTER_LINEAR
- borderMode: 가장자리 픽셀 확장 방식. 기본값은 cv2.BORDER_CONSTANT.
- borderValue: cv2.BORDER_CONSTANT일 때 사용할 상수 값. 기본값은 0.

어파인 변환과 투시 변환

- 투시 변환 예제 (찌그러진 명함 펴기)

```
src = cv2.imread('namecard.jpg')

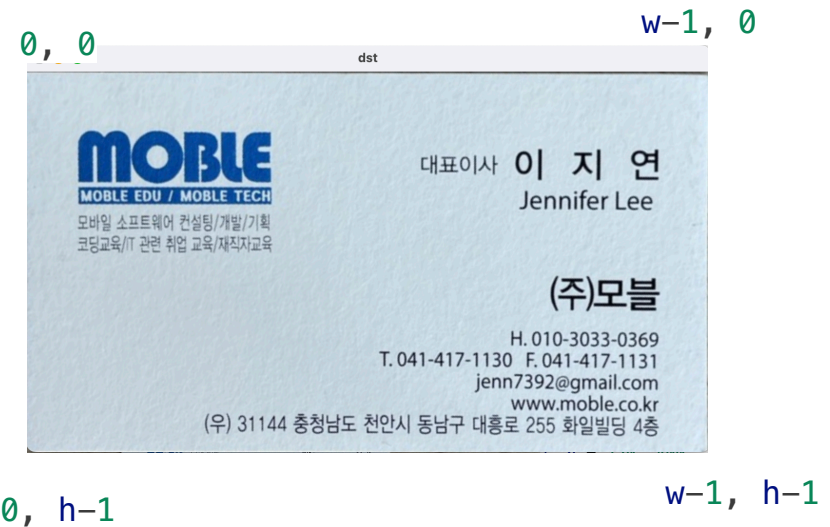
w, h = 720, 400
srcQuad = np.array([[325, 307], [760, 369], [718, 611], [231, 515]],
                    np.float32)
dstQuad = np.array([[0, 0], [w-1, 0], [w-1, h-1], [0, h-1]], np.float32)

pers = cv2.getPerspectiveTransform(srcQuad, dstQuad)

dst = cv2.warpPerspective(src, pers, (w, h))
```


어파인 변환과 투시 변환

투시 변환 예제 (찌그러진 명함 펴기) 실행 결과



기하학적 변환

6) 실전 코딩 : 문서 스캐너

[실전 코딩] 문서 스캐너

■ 문서 스캐너

- 카메라로 촬영한 문서 영상을 똑바로 펴서 저장해주는 프로그램

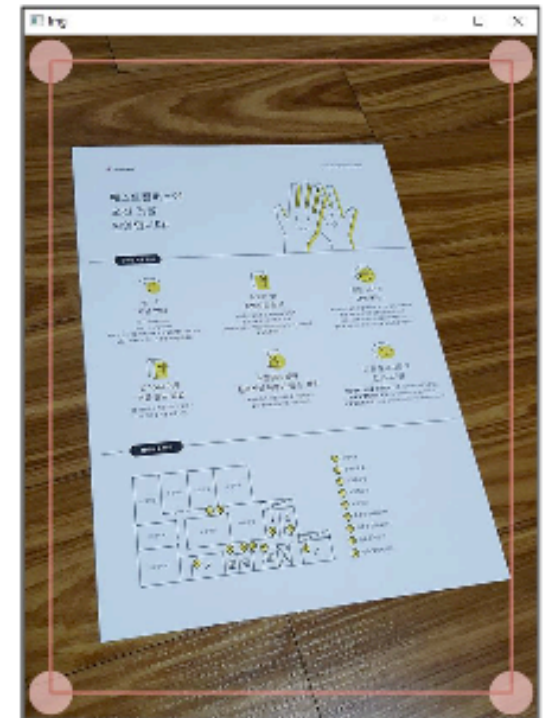


■ 구현 할 기능

- 마우스로 문서 모서리 선택 & 이동하기
- 키보드 ENTER 키 인식
- 왜곡된 문서 영상을 직사각형 형태로 똑바로 펴기 (투시 변환)

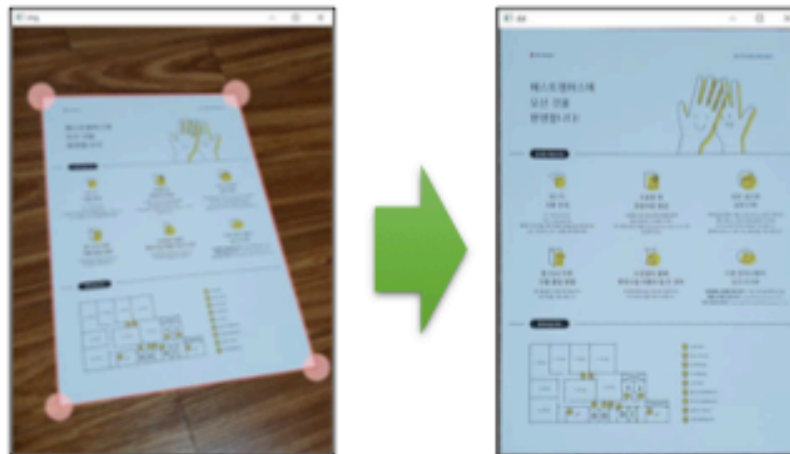
[실전 코딩] 문서 스캐너

- 마우스로 문서 모서리 선택 & 이동하기
 - 마우스 왼쪽 버튼이 눌린 좌표가
네 개의 모서리와 근접해 있는지를 검사
 - 특정 모서리를 선택했다면 마우스 드래그를 검사
 - 마우스 드래그 시 좌표 이동 & 화면 표시
 - 마우스 왼쪽 버튼이 떴을 때의 좌표를 기록



[실전 코딩] 문서 스캐너

- 왜곡된 문서 영상을 직사각형 형태로 똑바로 펴기 (투시 변환)
 - 네 개의 모서리 좌표를 순서대로 srcQuad 배열에 추가
 - dstQuad 배열에는 미리 정의한 A4 용지 크기의 네 모서리 좌표를 저장 (A4 용지 크기: 210x297cm)
 - srcQuad 점들로부터 dstQuad 점들로 이동하는 투시 변환 계산
 - 계산된 투시 변환 행렬을 이용하여 영상을 투시 변환하여 화면 출력



[실전 코딩] 문서 스캐너

```
import sys
import numpy as np
import cv2

def drawROI(img, corners):
    cpy = img.copy()

    c1 = (192, 192, 255)
    c2 = (128, 128, 255)

    for pt in corners:
        cv2.circle(cpy, tuple(pt.astype(int)), 25, c1, -1, cv2.LINE_AA)

    cv2.line(
        cpy,
        tuple(corners[0].astype(int)),
        tuple(corners[1].astype(int)),
        c2,
        2,
        cv2.LINE_AA,
    )
    cv2.line(
        cpy,
        tuple(corners[1].astype(int)),
        tuple(corners[2].astype(int)),
        c2,
        2,
        cv2.LINE_AA,
    )
```

[실전 코딩] 문서 스캐너

```
cv2.line(
    cpy,
    tuple(corners[2].astype(int)),
    tuple(corners[3].astype(int)),
    c2,
    2,
    cv2.LINE_AA,
)
cv2.line(
    cpy,
    tuple(corners[3].astype(int)),
    tuple(corners[0].astype(int)),
    c2,
    2,
    cv2.LINE_AA,
)

disp = cv2.addWeighted(img, 0.3, cpy, 0.7, 0)

return disp
```

[실전 코딩] 문서 스캐너

```
def onMouse(event, x, y, flags, param):
    global srcQuad, dragSrc, ptOld, src

    if event == cv2.EVENT_LBUTTONDOWN:
        for i in range(4):
            if cv2.norm(srcQuad[i] - (x, y)) < 25:
                dragSrc[i] = True
                ptOld = (x, y)
                break

    if event == cv2.EVENT_LBUTTONUP:
        for i in range(4):
            dragSrc[i] = False

    if event == cv2.EVENT_MOUSEMOVE:
        for i in range(4):
            if dragSrc[i]:
                dx = x - ptOld[0]
                dy = y - ptOld[1]

                srcQuad[i] += (dx, dy)

        cpy = drawROI(src, srcQuad)
        cv2.imshow("img", cpy)
        ptOld = (x, y)
        break
```


[실전 코딩] 문서 스캐너

```
# 입력 이미지 불러오기
src = cv2.imread("scanned.jpeg")

if src is None:
    print("Image open failed!")
    sys.exit()

# 입력 영상 크기 및 출력 영상 크기
h, w = src.shape[:2]
dw = 500
dh = round(dw * 297 / 210) # A4 용지 크기: 210x297cm

# 모서리 점들의 좌표, 드래그 상태 여부
srcQuad = np.array([[30, 30], [30, h - 30], [w - 30, h - 30], [w - 30, 30]], np.float32)
dstQuad = np.array([[0, 0], [0, dh - 1], [dw - 1, dh - 1], [dw - 1, 0]], np.float32)
dragSrc = [False, False, False, False]

# 모서리점, 사각형 그리기
disp = drawROI(src, srcQuad)

cv2.imshow("img", disp)
cv2.setMouseCallback("img", onMouse)
```

[실전 코딩] 문서 스캐너

```
while True:
    key = cv2.waitKey()
    if key == 13: # ENTER 키
        break
    elif key == 27: # ESC 키
        cv2.destroyWindow("img")
        sys.exit()

# 투시 변환
pers = cv2.getPerspectiveTransform(srcQuad, dstQuad)
dst = cv2.warpPerspective(src, pers, (dw, dh), flags=cv2.INTER_CUBIC)

# 결과 영상 출력
cv2.imshow("dst", dst)
cv2.waitKey()
cv2.destroyAllWindows()
```